

Extended Abstract

Motivation The finite amount of public data for training language models presents a critical challenge. This motivates our research question: Can we develop improved RL algorithms that take maximum advantage of limited data and compute? This paper uses the Countdown task as a testbed for novel algorithms designed to address this problem.

Method Our methods are inspired by human learning strategies and are built upon a supervised fine-tuned Qwen 2.5 0.5B model, using RLOO as a baseline. We propose two novel algorithms:

- Easy-to-hard with retries (ETHR): This algorithm organizes the training curriculum by dynamically identifying problems that are "too hard"—where the model fails to generate any correct solutions—and queuing them to be retried later.
- Dynamic data augmentation (DDA) for Countdown: This method creates a "bridge" from easy to hard problems by automatically generating new, more complex 4-number problems from the 3-number problems that the model solves correctly.

Implementation and results We trained our methods against vanilla RLOO for 2000 steps. Due to significant compute limitations, all experiments were run with a batch size of 1. Our primary finding is that our proposed algorithms did not significantly outperform the standard RLOO baseline. Surprisingly, RLOO achieved the best final performance on the difficult 4-number examples, the very tasks our methods were designed to improve. To investigate this, we defined a metric for "highly informative" examples. Our analysis revealed that while DDA was successful at producing informative 4-number examples, RLOO's strength may stem from its high exposure to informative 3-number examples.

Discussion and conclusion Our results were counter-intuitive and suggest that the simpler baseline's success may be due to reinforcing foundational skills on a high volume of informative, simple examples. This may be more critical than the more complex curriculum strategies we designed. The main limitation of our project was a lack of compute, which prevented longer training runs where data augmentation might show greater benefits. A key puzzle remains: why did generating what we identified as "good" data via DDA not translate to better evaluation performance? We believe this is a valuable direction for future work and hope our project inspires further research into what truly makes data valuable within a learning curriculum.

Curriculum Learning and Dynamic Data Augmentation for Countdown

Liam O’Carroll

Department of Computer Science
Stanford University
ocarroll@stanford.edu

Abstract

As the amount of public training data decreases, we investigate how to improve the data efficiency of RL algorithms using the Countdown task as a testbed. We propose two novel algorithms built upon a standard RLOO baseline: easy-to-hard with retries (ETHR), which adapts the problem sequence to the model’s current ability, and dynamic data augmentation (DDA) for Countdown, which generates intermediate "bridge" examples to ease the model into harder tasks. Ultimately, our experiments showed that neither algorithm significantly outperformed the vanilla RLOO baseline. Surprisingly, RLOO proved strongest on the most difficult 4-number problems—the tasks our methods were designed to improve. While our analysis using a "highly informative" data metric reveals that our DDA method successfully generated valuable training examples, this did not translate to better evaluation performance, a surprising outcome we hope leads to future work on more effective curriculum design.

1 Introduction

While there has been significant algorithmic innovation, breakthroughs in large language models over the past five years have mainly been driven by scale. This includes ever-increasing model sizes along with significantly more massive datasets constituting a nontrivial portion of all recorded human knowledge. However, there is a limit to how much data can be drawn from the internet; Villalobos et al. (2024) estimate that the available stock of public human text data will be exhausted between 2026 and 2032. This looming deadline motivates the critical question:

Can we develop improved RL algorithms which take maximum advantage of limited data and compute?

In this report, we study this problem using the Countdown task Gandhi et al. (2024) as a playground to test novel algorithms designed to tackle this problem. To start, we perform supervised fine-tuning on the Qwen 2.5 0.5B Base model, and then use RLOO Ahmadian et al. (2024) as a baseline with which to compare novel algorithms. Our algorithms are based off of the way humans learn; in particular, we take inspiration from the curriculum learning and data augmentation literature Narvekar et al. (2020); Wang et al. (2021); DeepSeek-AI et al. (2025); Gandhi et al. (2025), as further discussed in Section 2. To distinguish from previous work, we design algorithms with *fully automated task creation* which adapt to where the model is in the training trajectory; see Section 2 for further details.

Ultimately, we propose two novel algorithms:

- Easy-to-hard curriculum learning with retries, which combines a standard curriculum learning technique in the literature (training from easy examples to hard examples) with a novel way of determining which examples are hard.

- Dynamic data augmentation with automatic generation of a bridge between easy and hard examples. This algorithm separates the dataset into easy and hard examples, and while training on the easy examples first, creates “medium-difficulty” examples which serve as a bridge to the most difficult examples.

Ultimately, while our experimental results lead to some interesting insights discussed further in Section 5, our algorithms do not significantly outperform the standard RLOO baseline. Nonetheless, we hope our work inspires future research on improving performance on the Countdown task via curriculum learning and dynamic data augmentation. The rest of this report is organized as follows: In Section 2 we go over related work. We give our methods in Section 3, experimental setup in Section 4, and results in Section 5. We include discussion in Section 6 and conclude in Section 7.

2 Related Work

While there has been significant interest in curriculum learning recently, the question of what the best techniques are is far from settled—especially for LLMs. One recent work is DeepSeek-AI et al. (2025), which uses multi-stage training and cold-start data before applying RL. In Gandhi et al. (2025), the authors show that priming an LLM with examples containing specific reasoning behaviors enables substantial improvement for the Countdown task. However, their work requires human intervention to curate the dataset to contain these reasoning behaviors. A key open problem highlighted in the survey Narvekar et al. (2020) is *fully automated task creation*, where tasks are created “on-demand” to ideally best serve the model in its current state. Narvekar et al. (2020) note that there is limited work in this setting, and the work that does exist Narvekar et al. (2016); Silva and Costa (2018) is largely not tailored to fine-tuning LLMs. There has not been any work on automated task creation for the Countdown task in particular, to the best of our knowledge.

3 Method

In this section, we describe our novel algorithms for curriculum learning and dynamic data augmentation in Countdown. In Section 3.1, we briefly recap the RLOO algorithm Ahmadian et al. (2024) which our novel algorithms are based off of. We then describe our algorithms in Sections 3.2 and 3.3.

3.1 RLOO

RLOO is an improved version of the REINFORCE algorithm used in policy gradient methods. Its main goal is to make the learning process more stable by reducing how much the randomness of collected samples (variance) affects the policy updates.

It achieves this by using a clever baseline. For each action taken, the baseline is calculated as the average reward from all the other actions sampled from the policy in that same batch. The algorithm then updates the policy based on how much better or worse an action’s reward was compared to this leave-one-out baseline. Formally, the RLOO update is

$$\frac{1}{k} \sum_{i=1}^k \left[R(y_{(i)}, x) - \frac{1}{k-1} \sum_{j \neq i} R(y_{(j)}, x) \right] \nabla \log \pi(y_{(i)}, x) \text{ for } y_{(1)}, \dots, y_{(k)} \sim \pi_{\theta}(\cdot | x), \quad (1)$$

where x is the prompt and π_{θ} is the current policy.

3.2 Easy-to-hard with retries (ETHR)

Our first algorithm is *easy-to-hard with retries*. This algorithm is motivated by the key observation that the difficulty of the Countdown task *stems from the 4-number examples*. Indeed, the Countdown dataset <https://huggingface.co/datasets/Jiayi-Pan/Countdown-Tasks-3to4> is approximately evenly divided into 4-number and 3-number examples. However, our SFT baseline achieved evaluation reward 0.54 on the 3-number examples, whereas it achieved 0.24 on the 4-number examples. Thus, we propose Algorithm 1; for simplicity, we just give a single epoch, but Algorithm 1 can be looped for multiple epochs. Algorithm 1 differs from RLOO in two fundamental ways:

- First of all, unlike RLOO, it doesn't iterate over the dataset in a random order. Instead, drawing inspiration from the curriculum learning literature Narvekar et al. (2020) as well as how humans learn, it first iterates over the easier 3-number example before reaching the harder 4-number examples. The hope being that it is able to extract more information from the 4-number examples once they are reached since it is a stronger model at that point. This initial stage (iterating over the easy 3-number examples followed by the harder 4-number examples) encompasses Lines 1 through 12 in Algorithm 1.
- Furthermore, in this first stage, it dynamically determines what the "hard examples" are that it is not yet ready to learn from and leaves them for later. We determine a hard example by simply checking the rewards achieved from the batch of responses. If the rewards are all less than 1 (recall in Countdown the rewards are all 0, 0.1, or 1), then the model is not yet ready to learn from that example, and we place it in a queue to learn from later (see Lines 13 through 20 in Algorithm 1), hopefully when the model is ready. This is similar to how humans learn; if something is too hard, we wait until we have acquired more knowledge.

Algorithm 1: Easy-to-hard with retries for Countdown (single epoch)

Input: Dataset D_3, D_4 containing 3-number and 4-number examples from the Countdown task, parameter k associated with RLOO update (1), initial model parameters θ

```

1 Let  $Q_3$  denote an empty queue
2 for  $j = 1, 2, \dots, |D_3|$  do
3   Let  $x_j$  denote  $j$ -th prompt in  $D_3$ 
4    $y_{(1)}, \dots, y_{(k)} \sim \pi_{\theta}(\cdot | x_j)$ 
5   if  $\max_{1 \leq i \leq k} R(y_{(i)}, x_j) < 1$  then insert  $x_j$  into the queue  $Q_3$ 
6   Perform RLOO update (1) using  $y_{(1)}, \dots, y_{(k)}$ 
7 Let  $Q_4$  denote an empty queue
8 for  $j = 1, 2, \dots, |D_4|$  do
9   Let  $x_j$  denote  $j$ -th prompt in  $D_4$ 
10   $y_{(1)}, \dots, y_{(k)} \sim \pi_{\theta}(\cdot | x_j)$ 
11  if  $\max_{1 \leq i \leq k} R(y_{(i)}, x_j) < 1$  then insert  $x_j$  into the queue  $Q_4$ 
12  Perform RLOO update (1) using  $y_{(1)}, \dots, y_{(k)}$ 
13 while  $Q_3$  is not empty do
14   Draw prompt  $x$  from  $Q_3$ 
15    $y_{(1)}, \dots, y_{(k)} \sim \pi_{\theta}(\cdot | x)$ 
16   Perform RLOO update (1) using  $y_{(1)}, \dots, y_{(k)}$ 
17 while  $Q_4$  is not empty do
18   Draw prompt  $x$  from  $Q_4$ 
19    $y_{(1)}, \dots, y_{(k)} \sim \pi_{\theta}(\cdot | x)$ 
20   Perform RLOO update (1) using  $y_{(1)}, \dots, y_{(k)}$ 

```

3.3 Dynamic data augmentation (DDA) for Countdown

Our next algorithm introduces dynamic data augmentation for Countdown, which we give in Algorithm 2. For simplicity we give a single epoch, but it can be run multiple times. It proceeds as follows:

- As in easy-to-hard with retries (Algorithm 1), Algorithm 2 first iterates over the easier 3-number examples before reaching the 4-number examples, with the same justification as before. However, this time it aims to create a "bridge" between the easy 3-number examples and the hard 4-number examples via dynamic data augmentation. In particular, in Line 9, if a reward of 1 was achieved on the current prompt x_j (meaning we got it right), we create new "easy" 4-number examples based on our correct answer to the problem with x_j . As an example, suppose the prompt x_j has target 35 where the numbers are $[2, 54, 17]$, and it correctly gets the equation $54 - (17 + 2)$. Then we can generate new 4-number examples by appending different operations to this example, for example:

- Append addition: New target is 50 and numbers are [2, 54, 17, 15]
- Append subtraction: New target is 30 and numbers are [2, 54, 17, 5]
- Append multiplication: New target is 70 and numbers are [2, 54, 17, 2]
- Append division: New target is 5 and numbers are [2, 54, 17, 7]
- In the actual code, for each operation, we randomly generate a fourth number to append with this operation. These examples are then placed into a queue and drawn from in future iterations at *data augmentation steps*. In the actual code, the data augmentation steps are chosen to be equally spaced every, e.g., 3 to 4 steps in the future so that the example hopefully remains "fresh" in the model's memory.
- The motivation behind this algorithm is to "ease" into the difficult 4-number examples by first training on the easiest 4-number examples possible—they only differ from 3-number examples which the model correctly solved previously by a single operation and number. Thus, hopefully having trained on these easier four number examples, when the model reaches the brand new 4-number examples in Line 11, it is much stronger at these harder examples and ready to learn from them. Another advantage of this algorithm is its ability to create data on the fly; you can train on a much larger dataset than your base dataset.

Algorithm 2: Dynamic data augmentation for Countdown (single epoch)

Input: Dataset D_3, D_4 containing 3-number and 4-number examples from the Countdown task, parameter k associated with RLOO update (1), initial model parameters θ , number of 4-number examples to generate at each generation step m

```

1 Let  $Q$  denote an empty queue
2 for  $j = 1, 2, \dots, |D_3|$  do
3   if at data augmentation step then
4     Draw  $x_j$  from  $Q$ 
5   else
6     Let  $x_j$  denote  $j$ -th prompt in  $D_3$ 
7      $y_{(1)}, \dots, y_{(k)} \sim \pi_{\theta}(\cdot | x_j)$ 
8     Perform RLOO update (1) using  $y_{(1)}, \dots, y_{(k)}$ 
9     if  $\max_{1 \leq i \leq k} R(y_{(i)}, x_j) = 1$  and not at data augmentation step then
10      Generate new 4-number examples  $x'_1, \dots, x'_m$  and place in  $Q$ 
11 for  $j = 1, 2, \dots, |D_4|$  do
12   Let  $x_j$  denote  $j$ -th prompt in  $D_4$ 
13    $y_{(1)}, \dots, y_{(k)} \sim \pi_{\theta}(\cdot | x_j)$ 
14   Perform RLOO update (1) using  $y_{(1)}, \dots, y_{(k)}$ 

```

4 Experimental Setup

We train vanilla RLOO (Section 3.1), ETHR, and DDA on the same training dataset consisting of 500 3-number examples and 500 4-number examples from the Countdown dataset <https://huggingface.co/datasets/Jiayi-Pan/Countdown-Tasks-3to4>. To ensure fair comparisons, we run all methods for 2000 steps. For RLOO, this corresponds to exactly 2 epochs, whereas for the other methods, it is more than 1 epoch but less than 2 epochs due to the retries and data augmentation. We evaluate the methods during and after training on a held-out dataset consisting of 100 3-number examples and 100 4-number examples. We track the average reward on each of these individually every 200 steps, as well as the total average reward. All experiments are performed on AWS with GPUs.

Additionally, we evaluate two versions of the DDA Algorithm 2; a version which only augments with new addition and subtraction examples, and a version which augments with all possible operations (addition, subtraction, multiplication, division). As this was a response to initial results, we discuss the reason for this in Section 5.

Another metric I study is how many *highly informative* examples the model looked at during training. An example is highly informative for RLOO if the term

$$R(y_{(i)}, x) - \frac{1}{k-1} \sum_{j \neq i} R(y_{(j)}, x)$$

in the RLOO update (1) is large, since it means that the model was able to obtain a strong signal (i.e., drew both bad and good responses). In the context of Countdown where the rewards are 1, 0.1, and 0. I define an example x and the corresponding responses $y_{(1)}, \dots, y_{(k)} \sim \pi_{\theta}(\cdot | x)$ to be *highly informative* if at least one of $y_{(1)}, \dots, y_{(k)}$ achieves reward 1, and they don't all achieve reward 1.

Due to sharp limitations on compute (I had far less credits because I was working alone), I was only able to run with $k = 2$ and batch size 1.

5 Results

5.1 Quantitative Evaluation

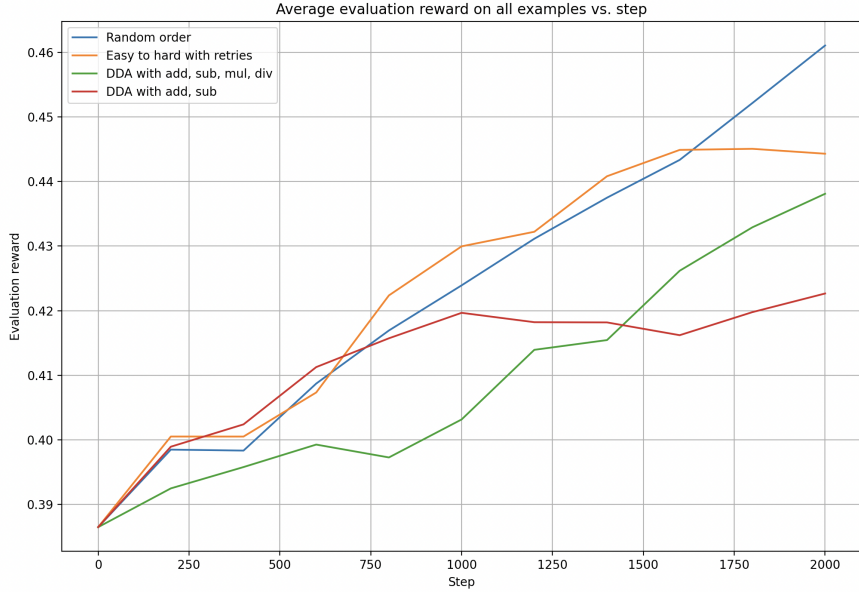


Figure 1: Comparison of methods on both 3-number and 4-number examples combined. The evaluation reward was computed on a held-out test set consisting of 200 examples, 100 3-number examples and 100 4-number examples. “Random order” is standard RLOO.

The evaluation reward on the held-out test set was computed every 200-steps for 100 3-number examples, 100 4-number examples, and both combined; see Figures 1, 2, and 3. We note that due to the large number of examples used for every evaluation, we believe these plots are not too noisy and indeed reflective of the actual model’s performance over time and at the end.

As discussed in Section 4, we include the basic RLOO algorithm, easy-to-hard with retries (ETHR), and two variants of dynamic data augmentation for Countdown (DDA); a variant which only augments with addition and subtraction operations and a version which augments with all possible operations (addition, subtraction, multiplication, division). As can be seen from Figure 1, the two best methods overall on the combined lengths were random order (namely vanilla RLOO) and easy-to-hard with retries. Surprisingly, vanilla RLOO ultimately performed the best on 4-number examples. This is surprising because the goal of my novel algorithms was specifically to improve the performance on the difficult four-number examples. Interestingly, easy-to-hard with retries did tend to outperform vanilla RLOO on easier 3-number examples. I perform qualitative analysis to try to explain these surprising results in the next section.

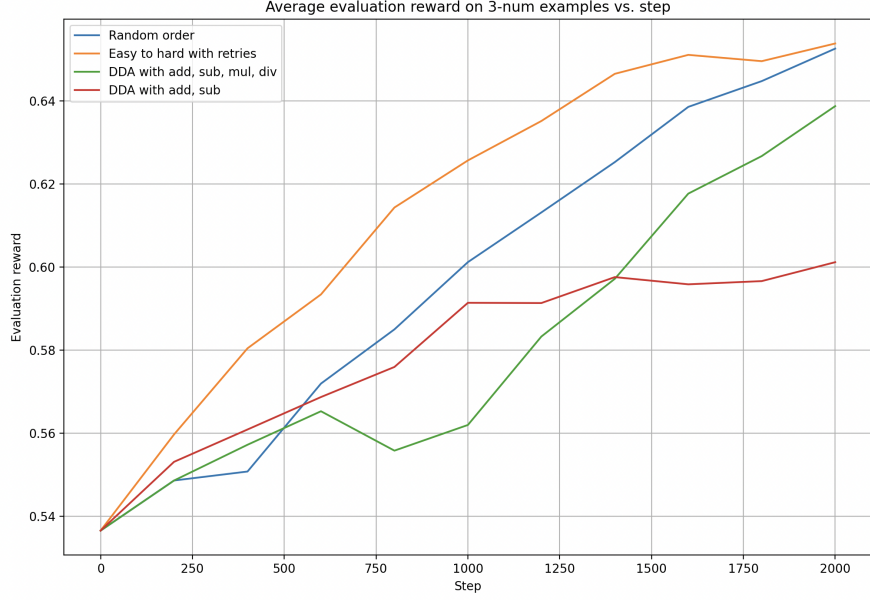


Figure 2: Comparison of methods on 3-number examples. The evaluation reward was computed on a held-out test set consisting of 100 3-number examples. “Random order” is standard RLOO.

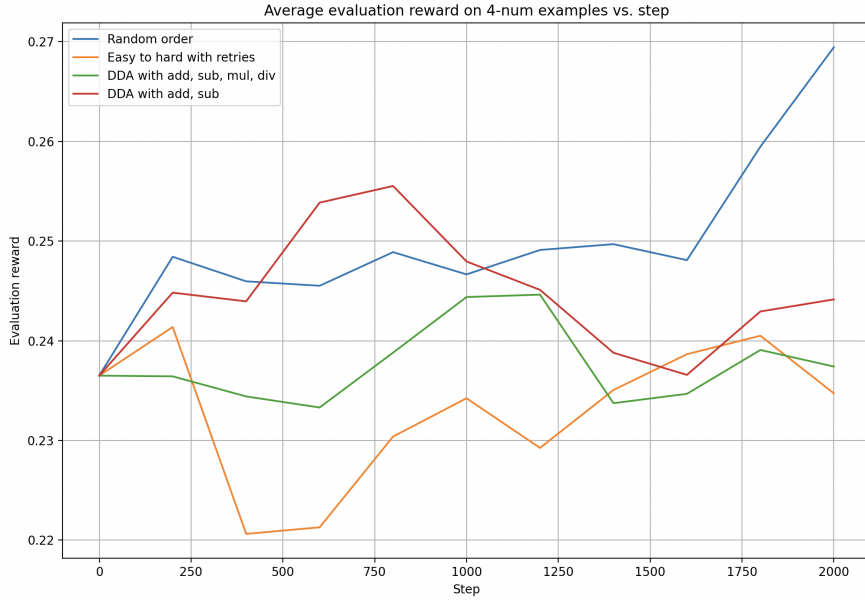


Figure 3: Comparison of methods on 4-number examples. The evaluation reward was computed on a held-out test set consisting of 100 4-number examples. “Random order” is standard RLOO.

5.2 Qualitative Analysis

Why did vanilla RLOO perform just as well as DDA and easy-to-hard with retries overall? Why did vanilla RLOO outperform these methods on the hard 4-number examples which I had specifically tailored to the hard 4-number examples? To answer these questions, I first studied a substantial number of sample generations from each method, which I included a few examples of in Appendix A for completeness. However, these sample generations were not markedly different. Thus, I developed a different metric which I introduced in Section 4; how informative an example is. I will not repeat

the definition of a highly informative example here as it is given in Section 4, but suffice to say it is an example which is very desirable for RLOO to learn from.

To determine whether an intervention was helpful or not, I computed the number of useful examples of each length for all of the methods. However, for my novel methods which introduced data augmentation or retries (DDA and ETHR), I specifically looked at the usefulness of the examples introduced by the intervention. Here are the results:

- For vanilla RLOO (random order), 14.4% of the 4-number examples were highly informative, and 31.6% of the 3-number examples were highly informative.
- For easy-to-hard with retries, of those examples which were retried, 26.8% of the 3-number examples retried were highly informative (when retried), and 7.9% of the 4-number examples were highly informative when retried.
- For DDA with all operations (add, sub, mul, div), 23.5% of the augmented data (which were all 4-number examples) were highly informative. Interestingly, this breaks down by the operation appended as follows:
 - Addition: 41.9% highly informative
 - Subtraction: 38.9% highly informative
 - Multiplication: 9.11% highly informative
 - Division: 5.33% highly informative
- As I explain below, this evaluation led me to run DDA with only add and sub augmented data. For this training run, 38.5% of the augmented data was highly informative, and it broke down by operation appended as follows:
 - Addition: 39.9% highly informative
 - Subtraction: 36.8% highly informative

My qualitative conclusion from this analysis is that for the countdown task when compute and data are limited, *it is beneficial to simply learn from lots of 3-number examples*. Indeed, vanilla RLOO sees the most 3-number examples in the 2000 steps out of any of the other methods (because DDA is only augmenting with 4-number examples; easy-to-hard with retries does send hard 3-number examples to the back of the line to be iterated over again, but there are more 4-number examples that get retried than 3-number examples since 4-number examples tend to be harder; the numbers were 380 4-number examples got retried to only 168 3-number examples). Furthermore, a large portion of these 3-number examples vanilla RLOO sees are highly informative: 31.6%. Compare this to easy-to-hard with retries, which saw less 3-number examples and a smaller percentage were highly valuable. With DDA with all operations, the addition and subtraction augmentations were often highly informative, but multiplication and division were not, likely because they were too hard. This is the reason I reran DDA with only addition and subtraction, which generated lots of highly informative 4-number examples. Despite this, somewhat paradoxically, vanilla RLOO still beat DDA with only addition and subtraction. The only reason I can think to explain this is that DDA overfit to the very easy 4-number examples it was seeing, and this didn't translate to performance on 4-number examples seen in the evaluation set. It could also be that I didn't have enough compute to run for long enough to see long-term trends. Still, I find this puzzling and think it would be a great direction for further work to nail down why DDA is getting highly informative 4-number examples but not outperforming vanilla RLOO on 4-number evaluations.

6 Discussion

I met several limitations and difficulties during the project. For one, I had limited credits due to working alone, and also only had access to AWS spot instance which would often cut out. Thus, it was challenging to run my experiment for too many steps. In terms of broader impacts, I hope my project pushes other researchers to study data curriculums and data augmentation for the Countdown task, perhaps to elucidate some questions I found puzzling. I discuss this further in the next section.

7 Conclusion

To conclude, I developed two novel algorithms for data curriculums and data augmentation for the Countdown task: easy-to-hard with retries and DDA. There are several directions for future research:

- I did an analysis in Section 5.2 involving highly informative examples to attempt to elucidate why certain algorithms were performing better. It revealed that 3-number examples were typically much more valuable than 4-number examples, which might partially explain the excellent performance of vanilla RLOO since it sees the most 3-number examples. At the same time, as I stated there, I find it very puzzling that vanilla RLOO scored the best on 4-number examples since DDA saw lots of highly informative 4-number examples. Is a better metric needed to understand the value of a data point?
- I was quite limited by compute—it would be very interesting to run larger scale experiments and see if any trends emerge. In particular, the advantages of data augmentation may become more pronounced under more epochs, as algorithms without data augmentation would only see stale examples.

All in all, I hope my project inspires other researchers to further study these important questions of how to get the most out of your data through data augmentation and curriculum learning. I believe this is a pressing question given we are quickly running out of new high quality data.

8 Team Contributions

Not applicable; I worked by myself.

References

- Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. 2024. Back to Basics: Revisiting REINFORCE Style Optimization for Learning from Human Feedback in LLMs. arXiv:2402.14740 [cs.LG] <https://arxiv.org/abs/2402.14740>
- DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojuan Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanjia Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen

- Zhang. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948 [cs.CL] <https://arxiv.org/abs/2501.12948>
- Kanishk Gandhi, Ayush Chakravarthy, Anikait Singh, Nathan Lile, and Noah D. Goodman. 2025. Cognitive Behaviors that Enable Self-Improving Reasoners, or, Four Habits of Highly Effective STaRs. arXiv:2503.01307 [cs.CL] <https://arxiv.org/abs/2503.01307>
- Kanishk Gandhi, Denise Lee, Gabriel Grand, Muxin Liu, Winson Cheng, Archit Sharma, and Noah D. Goodman. 2024. Stream of Search (SoS): Learning to Search in Language. arXiv:2404.03683 [cs.LG] <https://arxiv.org/abs/2404.03683>
- Sanmit Narvekar, Bei Peng, Matteo Leonetti, Jivko Sinapov, Matthew E. Taylor, and Peter Stone. 2020. Curriculum Learning for Reinforcement Learning Domains: A Framework and Survey. arXiv:2003.04960 [cs.LG] <https://arxiv.org/abs/2003.04960>
- Sanmit Narvekar, Jivko Sinapov, Matteo Leonetti, and Peter Stone. 2016. Source Task Creation for Curriculum Learning. In *Proceedings of the 2016 International Conference on Autonomous Agents & Multiagent Systems (Singapore, Singapore) (AAMAS '16)*. International Foundation for Autonomous Agents and Multiagent Systems, Richland, SC, 566–574.
- Felipe Leno Da Silva and Anna Helena Real Costa. 2018. Object-Oriented Curriculum Generation for Reinforcement Learning. In *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems (Stockholm, Sweden) (AAMAS '18)*. International Foundation for Autonomous Agents and Multiagent Systems, Richland, SC, 1026–1034.
- Pablo Villalobos, Anson Ho, Jaime Sevilla, Tamay Besiroglu, Lennart Heim, and Marius Hobbahn. 2024. Will we run out of data? Limits of LLM scaling based on human-generated data. arXiv:2211.04325 [cs.LG] <https://arxiv.org/abs/2211.04325>
- Xin Wang, Yudong Chen, and Wenwu Zhu. 2021. A Survey on Curriculum Learning. arXiv:2010.13166 [cs.LG] <https://arxiv.org/abs/2010.13166>

A Sample generations

See Section 5 for explanation.

Random order (vanilla RLOO)

Prompt: A conversation between User and Assistant. The user asks a question, and the Assistant so

User: Using the numbers [29, 84, 89], create an equation that equals 34. You can use basic arithm

Assistant: Let me solve this step by step.<|endoftext|>

Output: <think>

Let me try to find a way to reach 34.

First, let's look at numbers close to 34:

89-84 = 5, which could be useful as it's smaller than 34

29+5 = 34, but we still need to use 84 and 89...

Let's try working with 89 and 84:

89-84 = 5

Then 5+29 = 34!

This works! Let's verify:

89-84 = 5

5+29 = 34

We've reached our target!

</think>

<answer> (89-84)+29 </answer>

Target: 34

Nums: [29, 84, 89]

Score: 1.0

Easy-to-hard with retries

Prompt: A conversation between User and Assistant. The user asks a question, and the Assistant so

User: Using the numbers [79, 3, 14, 76], create an equation that equals 33. You can use basic arith

Assistant: Let me solve this step by step.<|endoftext|>

Output: <think>

Let me try to find a way to get to 33.

First, let me look for numbers close to 33 or factors/multiples that could be useful.

33 is close to 34, which could be reached by adding 1 to 32.

Looking at the numbers: 79, 3, 14, 76

Let me try working with 79 and 76 first:

$79 - 76 = 3$

This gives us a small number we could work with.

Now we have 3, 14, and 33 to work with.

$33 + 14 = 47$ (too big)

$47 - 3 = 44$ (too big)

Let me try a different approach:

$79 - 76 = 3$

$3 * 14 = 42$ (getting closer)

Let me work backwards from 33:

$33 + 14 = 47$

$47 - 79 = -32$ (not what we want)

One more try:

$79 - 76 = 3$

$3 * 14 = 42$

$42 + 33 = 75$ (too big)

Let me try multiplication with the larger numbers:

$76 - 3 = 73$

$73 - 14 = 59$ (too far)

New approach:

$79 - 76 = 3$

$3 * 14 = 42$

$42 + 33 = 75$ (too big)

Final attempt:

$76 - 3 = 73$

$73 - 79 = -6$

$-6 + 14 = -10$ (not what we want)

Let's try one last sequence:

$79 - 76 = 3$

$3 * 14 = 42$

$42 + 33 = 75$ (too big)

Wait! Let me try:

$79 - 76 = 3$

$3 + 14 = 17$

$17 + 33 = 50$ (too big)

Last attempt:

$76 - 3 = 73$

$73 - 79 = -6$

$-6 + 14 = -12$ (not what we want)

Found it!

$79 - 76 = 3$

$3 * 14 = 42$

$42 + 33 = 75$

$75 - 10 = 65$ (not what we want)

One final try:

$76 - 3 = 73$

$73 + 14 = 87$

$87 - 79 = 8$ (not what we want)

Last attempt:

$79 - 76 = 3$

$3 * 14 = 42$

$42 + 33 = 75$

$75 - 10 = 65$ (not what we want)

Found it!:

$79 - 76 = 3$

$3 * 14 = 42$

$42 + 33 = 75$

$75 - 10 = 65$

</think>

<answer> $(79 - 76) * 14 + 33$ </answer>

Target: 33

Nums: [79, 3, 14, 76]

Score: 0.1

DDA with all operations (add, sub, mul, div)

Prompt: A conversation between User and Assistant. The user asks a question, and the Assistant so

User: Using the numbers [73, 34, 60, 63], create an equation that equals 84. You can use basic ar

Assistant: Let me solve this step by step.<|endoftext|>

Output: <think>

Let me try to find a way to reach 84.

First, let me look for numbers close to 84:

73 is closest to 84, needing +11

60 and 63 are too far from 84

Let me try working with 73:

$73 + 34 = 107$ (too high)

$73 - 34 = 39$ (too low)

Let me try working with 60 and 63:

$60 + 63 = 123$ (too high)

$60 - 63 = -3$ (too low)

Let me try multiplication with the larger numbers:

$60 * 1.4 = 84$ (this is our target!)

So if I could get 1.4 from the other numbers...

Let me try a different approach:

$63 - 60 = 3$

$73 + 3 = 76$

$76 + 34 = 110$ (too high)

Let me try one more time:

63 - 60 = 3
73 + 34 = 107
107 - 34 = 73 (getting nowhere...)

Ah! Let me try:

60 + 34 = 94
94 - 73 = 21
21 + 63 = 84!

Found it! Working backwards to verify:

60 + 34 = 94
94 - 73 = 21
21 + 63 = 84

</think>

<answer> (60 + 34 - 73) + 63 </answer>

Target: 84

Nums: [73, 34, 60, 63]

Score: 1.0

DDA with add, sub

Prompt: A conversation between User and Assistant. The user asks a question, and the Assistant so

User: Using the numbers [45, 43, 83, 38], create an equation that equals 33. You can use basic ar

Assistant: Let me solve this step by step.<|endoftext|>

Output: <think>

Let me try to find a way to get to 33.

First, let's look at numbers close to 33:

45-43 = 2, which could be useful as it's small

83-38 = 45, which is too large

Let's work backwards from 33:

33 is 3 * 11, so maybe we can try to get these numbers.

Looking at the numbers again:

83-43 = 40, close to 33...

45-38 = 7, not very useful...

What if we try to get to 33 by first getting a larger number and then subtracting?

83-45 = 38, close to 33...

Then we could do: 83-45 = 38, then 38-38 = 0, not quite what we need...

Let's try multiplication with the smaller numbers:

45/38 $\approx$ 1.26, not helpful...

New approach:

83-43 = 40

40-45 = -5

-5+38 = 33!

Found it! Working backwards to verify:

83-43 = 40

40-45 = -5

-5+38 = 33

</think>

<answer> (83-43)-45+38 </answer>

Target: 33

Nums: [45, 43, 83, 38]

Score: 0.1